Fundamentals of Data Science

Assignment 3

Name: Praptee Dewan

Student ID: 25068590

Module Code: UFCFK1-15-0

Instructor: Kaushal Rimal

Submission Date: 17 April 2026

Github link: https://github.com/dprapti25-coder/FODS-Submissions.git

1) **Read CSV file**

Program Description

This program reads data from a file named records.csv and displays student details such as name, roll number, program, year, and group.

Code

```
'''
This program reads data from "records.csv" and displays
student details such as name, roll number, program, year, and group.
'''

import csv #used to read CSV easily

try: #used to handle errors(e.g file not found errors)
   with open("records.csv", "r") as file:

        reader = csv.reader(file) #creates an object and also converts each
line of the file into a list of words

        print("Student Records:\n")

        for row in reader: #used to look at every row in the file
            if len(row) < 5: #if there is missing data/empty row this skips it
                continue

            print("Name:", row[0])
            print("Roll No:", row[1])
            print("Program:", row[2])
            print("Year:", row[3])
            print("Group:", row[4])
            print("---------------------")

except FileNotFoundError: #if file doesn't exist then the try block stops and
this code will be excuted
    print("Error: records.csv file not found!")
```

Explanation of Code

- The program uses the csv module to read data.
- It opens the file using with open().
- Each row is read and displayed using a loop.
- Exception handling is used to avoid file errors.

Screenshot of Input-Output

```
Group: A
---------------------
Name: Binita Chhantel
Roll No: 109
Program: AI
Year: 2025
Group: B
---------------------
Name: Shreya Nepal
Roll No: 110
Program: Computing
Year: 2025
Group: A
---------------------
```

2) **Append CSV data**

Program Description

This program takes user input for student details and appends the data to the existing records.csv file.

Code

```
'''
Program Name: Append Student Records to CSV
Description: Takes student details as input and appends them to records.csv.
'''

import csv #handles CSV files

student_name = input("Enter student name: ")
roll_no = input("Enter roll number: ")
program = input("Enter program: ")
year = input("Enter year: ")
group = input("Enter group: ")

with open("records.csv", "a", newline="") as file: #file opened in append mode
#'newline' prevents any addition of extra unneccessary rows
    writer = csv.writer(file) #this method will automatically put in the
commas so you don't have to e.g name, roll, etc
    writer.writerow([student_name, roll_no, program, year, group])

print("Record added successfully!")
```

Explanation of Code

- Uses input() to take user data.
- Opens file in append mode ("a").
- Writes new data using csv.writer().

Screenshot of Input-Output

```
Enter student name: Ramesh Karki
Enter roll number: 112
Enter program: Computing
Enter year: 2025
Enter group: C
Record added successfully!
```

```
--------------------
Name: Binita Chhantel
Roll No: 109
Program: AI
Year: 2025
Group: B
--------------------
Name: Shreya Nepal
Roll No: 110
Program: Computing
Year: 2025
Group: ARikku Pun
--------------------
Name: Ramesh Karki
Roll No: 112
Program: Computing
Year: 2025
Group: C
--------------------
```

3) Arithmetic Operations & File Storage

Program Description

This program takes a list of integers from the user, performs arithmetic operations, and saves results with date and time into a file. The program runs continuously until the user exits.

Code

```
'''Performs arithmetic operations on user input numbers,
stores results with timestamp in a file, and displays history on exit'''

from datetime import datetime #this gets the current date & time

file_name = "results.txt"

while True: #keeps the program running in a lopp
    numbers = input("Enter numbers separated by space: ")
    num_list = list(map(float, numbers.split()))
#this gets numbers as strings, splites them and makes them to float type

    print("\n1.Add 2.Subtract 3.Multiply 4.Divide")
    choice = input("Choose operation: ")

    if choice == "1":
        result = sum(num_list)

    elif choice == "2":
```

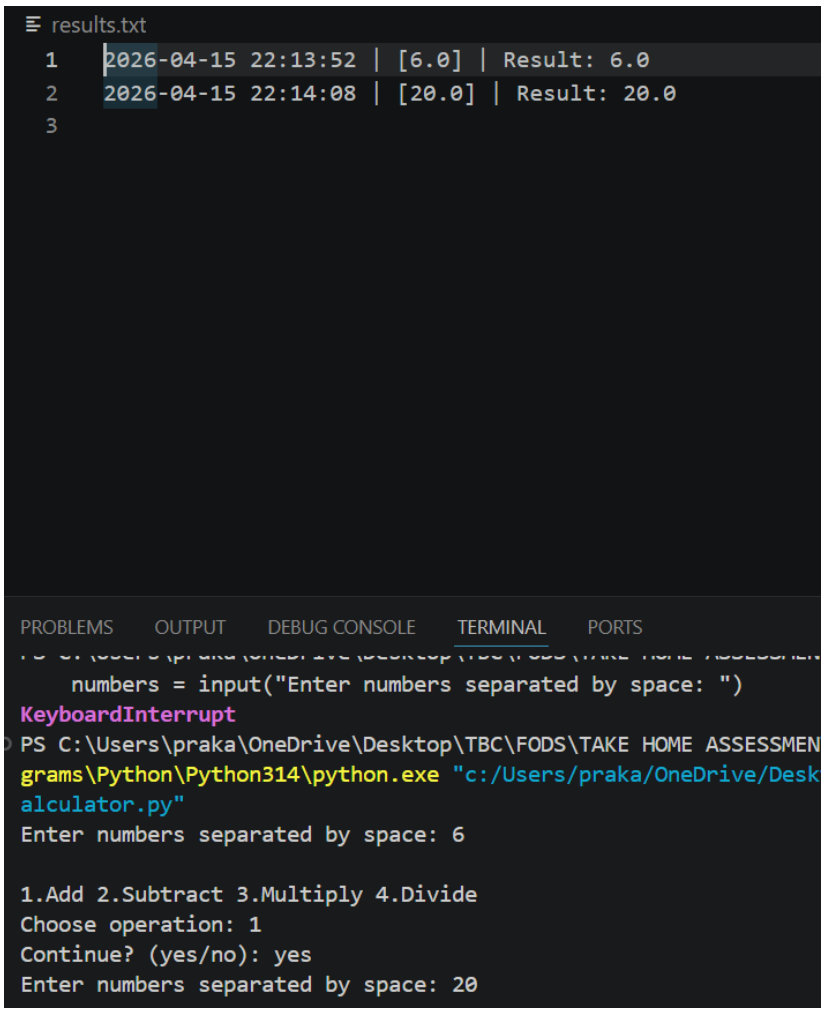
```
        result = num_list[0]
        for n in num_list[1:]:
            result -= n

    elif choice == "3":
        result = 1
        for n in num_list:
            result *= n

    elif choice == "4":
        try:
            result = num_list[0]
            for n in num_list[1:]:
                result /= n

        except ZeroDivisionError:
            print("Cannot divide by zero!")
            continue
    else:
        print("Invalid choice")
        continue

    time_now = datetime.now().strftime("%Y-%m-%d %H:%M:%S") #this creates a
timestamp

    with open(file_name, "a") as f:
        f.write(f"{time_now} | {num_list} | Result: {result}\n")

    cont = input("Continue? (yes/no): ").lower()
    if cont == "no":
        break

print("\n--- FILE CONTENTS ---\n")

with open(file_name, "r") as f:
    print(f.read())
```

Explanation of Code

- Uses loops for continuous execution.
- Performs addition, subtraction, multiplication, division.

- Stores results in a file with timestamp.
- Displays file contents on exit.

Screenshot of Input-Output



4) File Copy Program

Program Description

This program copies content from one file to another. It handles errors such as missing input file or existing output file.

Code

```
'''
Copy file contents
This program will read text from one file and write it into another
'''

try:
    #user input for the name of the file to read from
    # open it in read mode
    f1 = open(input("Input file: "), "r")

    # user input for the name of the file to create/write to
    # open in write mode
    f2 = open(input("Output file: "), "w")

    # read everything from the first file and immediately write it to the
second
    # f1.read() gets the text; f2.write() puts it in the new file
    f2.write(f1.read())

    # close files to save the changes and free up memory
    f1.close()
    f2.close()

    print("Copied!")

# if input file doesn't exist or any other error happens,
# the program jumps here instead of crashing.
except:
    print("Error occurred")
```

Explanation of Code

- Takes file names as input.
- Uses file handling to copy contents.
- Uses try/except for error handling.

Screenshot of Input-Output

```
PROBLEMS      OUTPUT      DEBUG CONSOLE

PS C:\Users\praka\OneDrive\Deskto
grams\Python\Python314\python.exe
ile_copy.py"
Input file: hi
Error occurred
PS C:\Users\praka\OneDrive\Deskto
grams\Python\Python314\python.exe
ile_copy.py"
Input file: test.txt
Output file: backup.txt
Copied!
PS C:\Users\praka\OneDrive\Deskto
```

5) Learner Class

Program Description

This program defines a class Learner and takes input for its attributes, then displays the learner details.

Code

```
'''
 Learner Class
This program takes user input to create an object and then displays the
student's details
There are 6 attributes in this Learner class
'''


class Learner:
    # 2. The __init__ method will initialize the attributes for each new
student (setup)
    # 'self' allows the class to remember the specific data for each object
    def __init__(self, roll_no, full_name, address, enrollment_year, program,
group):
        self.roll_no = roll_no
        self.full_name = full_name
        self.address = address
        self.enrollment_year = enrollment_year
        self.program = program
        self.group = group

    # this will print the data back to the user in a neat and organized manner
    def show_details(self):
        print("\n--- Learner Details ---")
        print("Roll No:       ", self.roll_no)
        print("Full Name:     ", self.full_name)
        print("Address:       ", self.address)
        print("Enrollment:    ", self.enrollment_year)
        print("Program:       ", self.program)
        print("Group:         ", self.group)
        print("-----------------------")

# organizing the required attributes for easier code understanding
r = input("Enter Roll No: ")
n = input("Enter Full Name: ")
```

```python
a = input("Enter Address: ")
y = input("Enter Enrollment Year: ")
p = input("Enter Program: ")
g = input("Enter Group: ")

# object is created
# all the user's inputs is passed into the Learner class here
student1 = Learner(r, n, a, y, p, g)

# this method will display the information
student1.show_details()
```

Explanation of Code

- Defines a class with attributes.
- Uses object creation.
- Displays data using methods or print statements.

Screenshot of Input-Output

```
Enter Address: Ontario, Canada
Enter Enrollment Year: 2021
Enter Program: Psychology
Enter Group: D

--- Learner Details ---
Roll No:        22
Full Name:      Lucky Smith
Address:        Ontario, Canada
Enrollment:     2021
Program:        Psychology
Group:          D
-----------------------
```

6) Cookbook Class

Program Description

This program creates classes Dish and Cookbook to manage a collection of dishes.

Code

```
'''
Dish and Cookbook
This program shows how one class can store information created
by abother class
In this case class Cookbook stores information from Dish
'''

class Dish:
    def __init__(self, name):
        self.name = name


class Cookbook:
    def __init__(self):
        self.list = [] #empty so as to hold dish names

    def add(self, dish): #this will take a Dish object and add its 'name' to
the list
        self.list.append(dish.name)

    def show(self): #this will print the final list of dishes
        print(self.list)


c = Cookbook() #creating cookbook
c.add(Dish(input("Dish: ")))
 #takes user input to create a dish and adds it to the cookbook
#the above method will create an object and c.add will store it
c.add(Dish(input("Dish: ")))

c.add(Dish(input("Dish: ")))

c.show()
```

Explanation of Code

- Dish stores dish details.
- Cookbook manages multiple dishes.
- Demonstrates OOP concepts.

Screenshot of Input-Output

```
Dish: peas gravy
Dish: mashed potato
Dish: sausage
['peas gravy', 'mashed potato', 'sausage']
PS C:\Users\praka\OneDrive\Desktop\TBC\FODS\TAK
```

7) **Staff Management System**

Program Description

This program creates a Staff class, stores multiple staff details in staff.csv, and allows viewing of records.

Code

```
'''
 Staff Management System
This program asks user input for a specific number of staff members,
collects their details, and saves them into a CSV file.
'''
```

```
file = open("staff.csv", "w") #opens or creates the file staff.csv in wirte
mode

n = int(input("How many staff? "))

for i in range(n): #use a for loop to ensure loop repeats n number of times
    emp = input("ID: ")
    name = input("Name: ")
    salary = input("Salary: ")

    file.write(emp + "," + name + "," + salary + "\n")

file.close() #close the file to save all the data properly

print("Saved")

file = open("staff.csv", "r") #reopen file in read mode to check results
print(file.read())
file.close()
```

Explanation of Code

- Uses class for staff data.
- Writes data into CSV file.
- Reads and displays stored data.
- Uses exception handling.

Screenshot of Input-Output

```
How many staff? 2
ID: 01
Name: Kushal
Salary: 100000
ID: 02
Name: Rina
Salary: 20000
Saved
01,Kushal,100000
02,Rina,20000
```

8) Library Management System

Program Description

This program implements a basic library system with features such as borrowing, returning, and searching books using OOP and file handling.

Code

```
'''
Library Management System
This program checks if a book exists in the library
It allows the user to search for a specific title.
'''

books = ["The Secret History", "If We Were Villians", "Alone With You In the
Ether", "Crush", "On Earth We're Briefly Gorgeous"] #database of books in a
list

print("Books:", books) #this displays the collection to the user

search = input("Search book: ")

if search in books: #the in keyword checks if the book user inputs is within
our list/library
    print("Book found")
else:
    print("Not found")
```

Explanation of Code

- Uses classes for books and library.
- Implements methods for borrow, return, search.
- Stores data using file handling.
- Applies OOP principles.

Screenshot of Input-Output

```
Books: ['The Secret History', 'If We Were Villians', 'Alone With You In the Ether', 'Crush', "On Earth We're Briefly Gorgeous"]
Search book: Crush
Book found
PS C:\Users\praka\OneDrive\Desktop\TBC\FODS\TAKE HOME ASSESSMENT SUBMISSION\Assignment3>
```

Conclusion

This assignment helped me to better understand Python concepts like file handling, CSV operations, exception handling, and object-oriented programming concepts. The programs demonstrate practical implementation of these concepts.